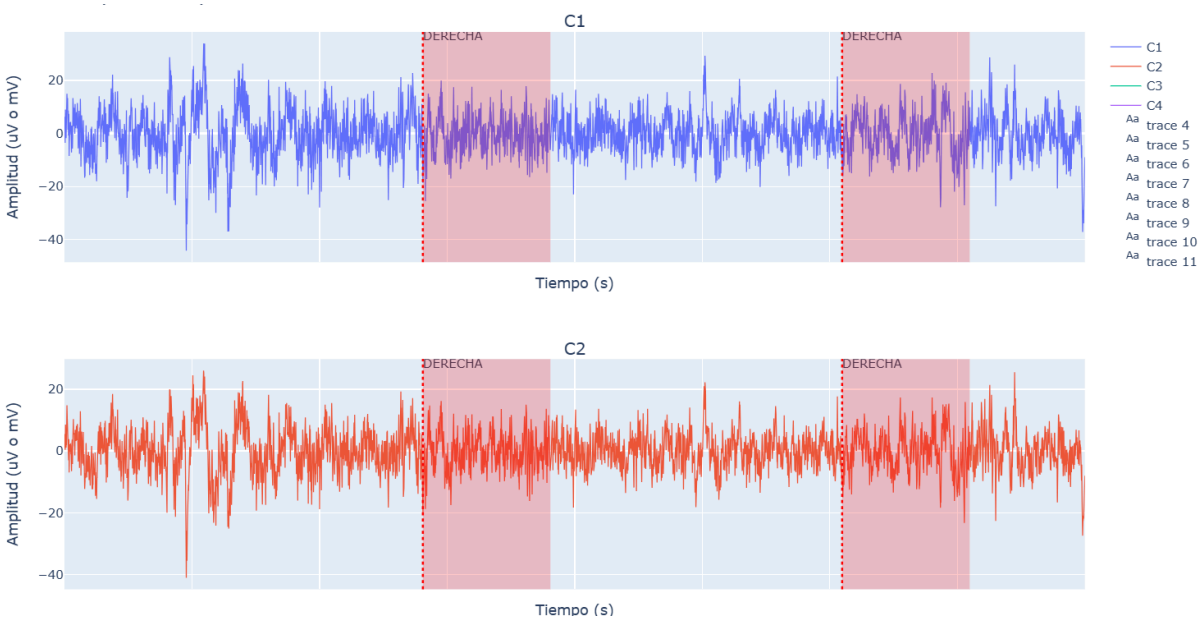
Trabajo Integrador Final

Librería para el procesamiento y análisis de señales de EEG, ECG y EMG



UC: Programación Digital Avanzada

Docentes:

- Lucas Baldezzari
- Emiliano Alvarez

Integrantes:

- Sofía de Bérail
- Valentin Ferrando
- Victoria Maneiro

Sede:

- ITR Suroeste, Fray Bentos, Uruguay

Fecha: 30/06/2025

1 Introducción

El procesamiento de señales fisiológicas—tales como EEG, ECG y EMG—es esencial en ingeniería biomédica y medicina para extraer información diagnóstica y de investigación. Generadas por la actividad eléctrica cerebral, cardíaca y muscular, estas señales suelen verse afectadas por ruido (interferencias de red eléctrica, movimiento de electrodos) y artefactos (parpadeos, contracciones no deseadas), por lo que su tratamiento contempla el filtrado digital mediante filtros de paso bajo, paso alto y notch para atenuar componentes no deseados; la detección de eventos, como picos R en ECG, ondas alfa/beta en EEG o activaciones musculares en EMG; la extracción de características temporales (media, varianza, RMS) y frecuenciales (bandas de EEG, variabilidad de frecuencia cardíaca, contenido espectral de EMG); y, finalmente, la visualización básica a través del trazado de series temporales y espectrogramas para una interpretación cualitativa.

En este Trabajo Integrador Final, se desarrollará una librería en Python, basada en NumPy, Pandas y Matplotlib, empleando Programación Orientada a Objetos para lograr una arquitectura modular y escalable.

2 Objetivos

2.1 Objetivo general

Diseñar, desarrollar e implementar una librería para el procesamiento, análisis y visualización de señales fisiológicas de EEG, ECG y EMG, utilizando Programación Orientada a Objetos (POO) en Python.

2.2 Objetivos específicos

- Aplicar NumPy y Pandas para el tratamiento eficiente de datos biomédicos.
- Utilizar Matplotlib para la visualización de señales en el dominio del tiempo y la frecuencia.
- Implementar técnicas de procesamiento de señales, incluyendo filtrado, detección de picos y análisis espectral, entre otros.
- Visualizar correctamente las señales en diferentes dominios utilizando Matplotlib.
- Extraer características relevantes de cada tipo de señal.
- Investigar y utilizar herramientas para el procesamiento y análisis de señales fisiológicas.
- Realizar pruebas sistemáticas para asegurar la calidad del software desarrollado.
- Realizar gestión de versiones utilizando Git y GitHub.
- Documentar el código y el proceso de desarrollo de la librería.

3 Materiales y herramientas

Para el desarrollo e implementación del Sistema de Matriculación se utilizaron los siguientes recursos:

- **Lenguaje de programación:** Python 3.12.10
- **Entorno de desarrollo integrado (IDE):** Visual Studio Code

4. Estructura del sistema

4.1 Estructura del sistema

Clase Info

La clase Info centraliza y almacena en un único objeto todos los metadatos relevantes asociados al registro de señales fisiológicas, incluyendo nombres y tipos de canales, canales marcados como “malos”, frecuencia de muestreo, descripción del experimento, nombre del experimentador e información adicional del sujeto, y proporciona una interfaz robusta para su validación, acceso y manipulación.

Además de inicializar y validar estos datos al momento de creación del objeto, la clase Info implementa métodos que le permiten comportarse como un diccionario, facilitando su integración con otros módulos del sistema.

Entre sus métodos se destacan:

- __contains__: permite verificar si una clave está presente en el objeto.
- __getitem__: habilita el acceso directo por clave como en un diccionario.
- __len__: devuelve la cantidad de elementos registrados.
- get: accede al valor de una clave sin riesgo de error si no existe.
- keys: devuelve las claves almacenadas.
- items: permite iterar sobre pares clave–valor.
- rename_channels: renombra de manera segura los canales existentes, garantizando que no se introduzcan duplicados ni errores de referencia.

Esta clase está diseñada para garantizar la coherencia de los metadatos desde el inicio, validando aspectos como la consistencia entre listas, la unicidad de los nombres de canales, la validez de los marcados como "malos", y la frecuencia de muestreo.

Clase Anotaciones

La clase Anotaciones permite almacenar y gestionar eventos fisiológicos temporales asociados a registros de señales. Centraliza la información de las anotaciones en tres listas paralelas: onset (tiempo de inicio), duration (duración del evento) y description (descripción textual del evento), y genera internamente un DataFrame de Pandas para facilitar su manipulación, consulta y persistencia en archivos CSV.

Entre sus métodos principales, la clase Anotaciones incluye funcionalidades que permiten agregar nuevas anotaciones, asegurando que no se repitan las descripciones y que el nuevo tiempo de inicio (onset) sea posterior al último registrado; eliminar eventos mediante el índice o una descripción exacta; obtener una copia completa de todas las anotaciones registradas; y buscar eventos que coincidan con una determinada descripción. También permite cargar anotaciones desde archivos CSV estructurados mediante el método load, y guardar los datos actuales en disco con el método save.

Clase RawSignal

La clase RawSignal representa una señal fisiológica cruda (por ejemplo, EEG, EMG, ECG, etc.) almacenada en un arreglo NumPy bidimensional, donde cada fila corresponde a un canal y cada columna a una muestra temporal. Su propósito es proporcionar una estructura robusta y flexible para almacenar, acceder, procesar y visualizar señales biológicas adquiridas digitalmente, integrando además metadatos (Info) y eventos experimentales (Anotaciones).

El constructor permite inicializar un objeto RawSignal a partir de una matriz de datos y su frecuencia de muestreo, validando que la forma y el tipo de datos sean adecuados, que la frecuencia no sea negativa y que, en caso de incorporar metadatos (Info) o anotaciones (Anotaciones), estos estén correctamente sincronizados y no excedan la duración de la señal.

Una vez creado el objeto, la clase ofrece una serie de métodos para trabajar con las señales:

- get_data permite extraer un segmento de la señal en un rango temporal y sobre un subconjunto de canales, con opciones para rechazar canales que superen un umbral de amplitud y para retornar el vector de tiempos asociado.
- drop_channels elimina uno o más canales del conjunto, retornando un nuevo objeto RawSignal con los metadatos y anotaciones actualizados.
- crop extrae un segmento temporal de la señal y ajusta automáticamente las anotaciones que caen dentro de ese intervalo.
- describe calcula estadísticas descriptivas por canal (mínimo, cuartiles, mediana y máximo), devolviendo un DataFrame con los resultados.
- filter aplica un filtro pasabanda (Butterworth) y un filtro notch (IIR) para eliminar componentes no deseadas de la señal, retornando una nueva instancia filtrada.
- pick permite seleccionar un subconjunto de canales mediante nombres, índices o slices, devolviendo un nuevo objeto con solo esos canales.
- set_anotaciones asocia un nuevo objeto de tipo Anotaciones a la señal, validando que todos los eventos estén dentro de la duración total.
- plot genera una visualización gráfica del segmento de señal especificado, con opción a superponer las anotaciones como líneas o regiones resaltadas con sus respectivas descripciones.

Clase EEGSignal

La clase EEGSignal hereda de RawSignal y está diseñada para procesar señales EEG, facilitando la manipulación del esquema de referencia espacial y análisis frecuencial. Cuenta con el atributo reference que indica el tipo de referencia aplicada. Se implementaron métodos específicos, algunos de ellos son, set_reference_to(), que resta la señal del canal seleccionado a todas las demás para definir una nueva referencia espacial, y apply_laplacian(), que realiza un filtrado espacial tipo Laplaciano restando la señal promedio de los canales vecinos inmediatos a cada canal central, manteniendo intactos los canales extremos.

Clase ECGSignal

La clase ECGSignal hereda de RawSignal y está diseñada para procesar señales ECG, facilitando la detección automática de picos R y el cálculo de la frecuencia cardíaca. Cuenta con los atributos picos_r, que almacena los índices de los picos detectados, y frecuencia_cardiaca_bpm, que guarda el valor calculado de la frecuencia cardíaca. Se implementaron métodos específicos: detectar_picos_r(canal, umbral), que identifica los picos R considerando un umbral mínimo y distancia entre latidos, y calcular_frecuencia_cardiaca(), que calcula los intervalos RR a partir de los picos detectados y determina la frecuencia cardíaca en latidos por minuto (BPM). Estos métodos actualizan internamente los atributos correspondientes para facilitar el análisis posterior de la señal ECG.**Clase EMGSignal**

La clase EMGSignal hereda de RawSignal y está diseñada para procesar señales EMG, facilitando la detección automática de activaciones musculares. Cuenta con los atributos umbral_activacion, que define el valor mínimo para considerar una activación muscular, y activaciones, que almacena listas de índices donde la señal supera el umbral establecido en cada canal. Se implementó el método específico detectar_activaciones(), que recorre todos los canales y registra automáticamente los puntos donde la señal excede el umbral definido. Este método actualiza internamente el atributo activaciones, facilitando análisis posteriores relacionados con la actividad muscular.

Módulo utils

Además de las clases principales, se desarrolló un módulo auxiliar denominado utils.py, que agrupa funciones básicas y reutilizables y algunas de ellas empleadas en varios métodos del sistema. Este módulo contribuye a mantener el código más ordenado, legible y modular, facilitando su mantenimiento y futura expansión.

Entre las funciones implementadas se encuentra check_types(), ampliamente utilizada para validar que los argumentos coincidan con los tipos esperados, lanzando un TypeError si no lo hacen. También se incluye normalize_signal(), que ajusta una señal NumPy para que tenga media cero y desviación estándar uno; y

time_to_samples(), que convierte un tiempo dado en segundos al número de muestras correspondiente según la frecuencia de muestreo.

Si bien algunas de estas funciones (como normalize_signal() y time_to_samples()) actualmente se emplean en pocos métodos o aún no han sido integradas, su inclusión en el módulo responde a criterios de diseño robusto. Esto permite centralizar funcionalidades genéricas que podrán reutilizarse fácilmente en futuras mejoras, como la normalización previa al análisis espectral o la especificación de ventanas temporales expresadas en segundos.

4.3 Diagrama UML

El diagrama UML de la figura 1 modela el programa implementado construido bajo principios de POO, con una clara separación de responsabilidades entre clases y una estructura modular que favorece la escalabilidad. La organización de clases y sus relaciones reflejan distintos niveles de acoplamiento y dependencia funcional.

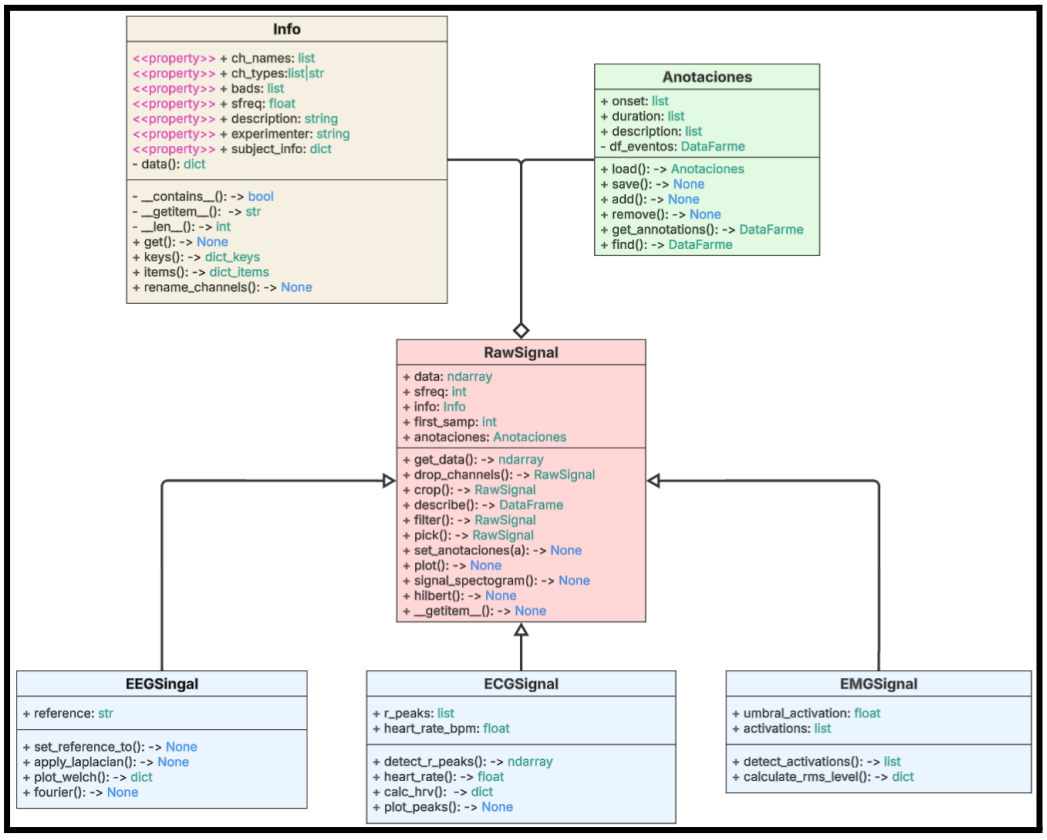


Figura 1: *Diagrama UML del sistema implementado.*

Relación entre RawSignal y Info: Agregación

La clase RawSignal mantiene una relación de agregación con la clase Info, ya que en su constructor recibe un objeto Info que reúne todos los metadatos de la señal (nombres y tipos de canales, frecuencia de muestreo, datos del sujeto, etc.), pero no crea esa instancia. Gracias a esta agregación, RawSignal puede usar métodos de Info para validar y acceder a la información de los canales sin duplicar esa lógica.

Relación entre RawSignal y Anotaciones: Agregación

La clase RawSignal mantiene también una agregación con la clase Anotaciones. Al instanciarse, RawSignal puede recibir un objeto Anotaciones que contiene onset, duration y description de cada evento, y lo almacena en su atributo anotaciones. Esta relación permite gestionar eventos asociados a la señal sin asumir responsabilidad sobre la creación o destrucción de las anotaciones.

Relación entre EEGSignal, ECGSignal, EMGSignal y RawSignal: Herencia

Las clases EEGSignal, ECGSignal y EMGSignal heredan de RawSignal, lo que significa que reutilizan todos los atributos y métodos de la clase base (acceso a datos, filtrado, recorte, visualización, etc.) y a su vez añaden comportamientos específicos.

5 Análisis y resultados

Se presentan las pruebas realizadas sobre los principales métodos de las distintas clases que conforman la librería, con el fin de verificar su correcto funcionamiento.

Clase Info

En la figura 2 se muestra la inicialización de un objeto Info con la ruta de metadatos y la creación de info_experimento pasando listas de canales, tipos, canales “bads”, frecuencia de muestreo, descripción, experimentador e información del sujeto. Se accedió a las propiedades ch_names, ch_types, bads, sfreq, description, experimenter y subject_info para verificar que reflejaran correctamente los valores

iniciales. Mediante la figura 3 se muestra el método __contains__() y en la figura 4 __getitem__() donde se comprobó la presencia y recuperación de claves válidas.

```
# ***** Generando un objeto nuevo *****
canales = [f"C{i+1}" for i in range(4)]

info_experimento = Info(
    ch_names= canales,
    ch_types= "emg",
    bads= ["C2"],
    sfreq= 512,
    description= "Registro EEG para análisis de patrones ERDS",
    experimenter= "MSc. BALDEZZARI Lucas",
    subject_info= {"edad": 29, "sexo": "M"}
    )
```

Figura 2 *Inicialización de clase Info.*

```
# ***** Usando el método __contains__() ****** (Verificamos si las keys existen)
print(f"\nEsta 'sfreq' en Info? : {'sfreq' in info_experimento} ")
print(f"Esta 'unknown_key' en Info? : {'unknown_key' in info_experimento}")
```

✓ 0.0s

```
Esta 'sfreq' en Info? : True
Esta 'unknown_key' en Info? : False
```

Figura 3: *Prueba del método contains().*

```
# ***** Usando el método __getitem__() ******
print(f"\nAccediendo al valor de 'sfreq' con []: {info_experimento['sfreq']}")
# print(f"Accediendo al valor de 'unknown_key' con []: {info["unknown_key"]}")

```

✓ 0.0s

```
Accediendo al valor de 'sfreq' con []: 512
```

Figura 4: *Prueba del método getitem().*

Clase Anotaciones

La figura 5 presenta la creación de un objeto Anotaciones, en la figura 6 la adición de eventos con add y en la figura 7 su eliminación mediante remove en ambas se verifica que get_anotaciones() devuelva un DataFrame con las columnas onset, duration y description.

```
# ***** Generando un objeto nuevo *****
onsets = [0.0, 5.0, 10.0]
durations = [1.0, 0.5, None]
descriptions = ["Inicio", "Evento A", "Evento B"]

eventos_experimento= Anotaciones(onset= onsets, duration= durations, description= descriptions)
```

Figura 5: *Inicialización de clase Anotaciones.*

```
# ***** Agregando una nueva anotación válida *****
eventos_experimento.add(new_onset= 15.0, new_duration= 2.0, new_description= "Evento C")

print("\n→ DataFrame tras agregar un nuevo evento:")
print(eventos_experimento.get_annotations())
```

✓ 0.0s

```
→ DataFrame tras agregar un nuevo evento:
   onset  duration description
0    0.0       1.0      Inicio
1    5.0       0.5    Evento A
2   10.0       NaN    Evento B
3   15.0       2.0    Evento C
```

Figura 6: *Prueba del método add().*

```
# ***** Eliminando una anotación por índice *****
eventos_experimento.remove(index= 1)

print("\n→ DataFrame tras eliminar el evento en índice 1:")
print(eventos_experimento.get_annotations())
```

✓ 0.0s

```
→ DataFrame tras eliminar el evento en índice 1:
   onset  duration description
0    0.0       1.0      Inicio
1   15.0       2.0    Evento C
```

Figura 7: *Prueba del método remove().*

Clase RawSignal

En la figura 8 se inicializa un objeto RawSignal.

```python
# ======================== Cargar datos y crear objeto RawSignal ========================

data_ecg = np.load("ecg/ecg/ecg.npy")

sfreq = 512

canales = [f"C{i+1}" for i in range(1)]

info_exp = Info(
    ch_names= canales,
    ch_types= "ecg",
    sfreq= sfreq,
    description="Registro ECG para FC",
    experimenter="MSc. BALDEZZARI Lucas",
    subject_info={"edad": 20, "sexo": "F"}
)

eventos_exp = Anotaciones.load("ecg/ecg/eventos_ecg.csv", sfreq= sfreq)

first_samp = 0

eeg_rawsignal = RawSignal(data= data_ecg, sfreq= sfreq, info= info_exp, anotaciones= eventos_exp, first_samp= first_samp)
```

Figura 8: *Inicialización de clase RawSignal.*

En la figura 9 se prueba get_data() para extraer segmentos temporales y subconjuntos de canales, validando las dimensiones de salida y el vector de tiempos. En la figura 10 se prueba drop_channels() donde se retiró un canal, mientras que crop() recortó la señal y ajustó las anotaciones fuera de rango.

```python
# ======================== Método get_data() ========================
print(">> Método get_data()")

# Obtener todos los datos
data_all = eeg_rawsignal.get_data()
print("data_all shape:", data_all.shape)

# Obtener un canal por nombre
data_c3 = eeg_rawsignal.get_data(picks="C1", start=1.0, stop=2.0)
print("data_c3 shape:", data_c3.shape)

# Obtener varios canales por nombre
data_mult = eeg_rawsignal.get_data(picks=["C1"], start=1.0, stop=2.0*5)
print("data_mult shape:", data_mult.shape)

# Obtener datos con vector de tiempo
data_t, times = eeg_rawsignal.get_data(times=True)
print("data_t shape:", data_t.shape, " | times shape:", times.shape)

# Obtener datos con rechazo de canales por amplitud pico a pico
data_rej = eeg_rawsignal.get_data(reject=200.0)
print("data_rej shape:", data_rej.shape)
```

✓ 0.0s

```
>> Método get_data()
data_all shape: (1, 388971)
data_c3 shape: (1, 512)
data_mult shape: (1, 4608)
data_t shape: (1, 388971)  | times shape: (388971,)
data_rej shape: (0, 388971)
```

Figura 9: *Prueba del método get_data().*

```python
# ======================== Método drop_channels() ========================
print("\n>> Método drop_channels()")

signal_dropped = eeg_rawsignal.drop_channels(['C1'])
print("Shape tras eliminar C1:", signal_dropped.get_data().shape)
print("Canales restantes:", signal_dropped.info['ch_names'])

# ======================== Método crop() ========================
print("\n>> Método crop()")

signal_cropped = eeg_rawsignal.crop(tmin=0.0, tmax=10.0)
print("Shape tras crop:", signal_cropped.get_data().shape)
```

✓ 0.0s

```
>> Método drop_channels()
Shape tras eliminar C1: (0, 388971)
Canales restantes: []

>> Método crop()
Shape tras crop: (1, 5120)
```

Figura 10: *Prueba del método drop_channels()..*

Clase EEGSignal

La figura 11 muestra pruebas de EEGSignal utilizando tres canales de ejemplo. En la figura 12 se ejecutó set_reference_to() usando nombre e índice de canal y se confirmó el cambio en data y en el atributo reference. En la figura 13 se probó apply_laplacian() con más de tres canales.

```python
# ======================== Cargar datos y crear objeto EEGSignal ========================

data_eeg = np.load("eeg/eeg/eeg_signal.np")

sfreq = 512

canales = [f"C{i+1}" for i in range(data_eeg.shape[0])]

info_eeg = Info(
    ch_names= canales,
    ch_types= "eeg",
    sfreq= sfreq,
    description="Registro EEG",
    experimenter="Dr. NEURO",
    subject_info={"edad": 35, "sexo": "M"}
)

eventos_eeg = Anotaciones.load("eeg/eeg/eventos_ejemplo.csv")

eeg = EEGSignal(data=data_eeg, sfreq=sfreq, info=info_eeg, first_samp=0, anotaciones=eventos_eeg)
```

Figura 11: *Inicialización de clase EEGSignal.*

```python
# ======================== Método cambiar_referencia_a() ========================
print("\n>> Prueba cambiar_referencia_a()")

# Mostrar referencia inicial
print("Referencia inicial:", eeg.referencia)

# Cambiar a un canal válido por nombre
eeg.cambiar_referencia_a("C1")
print("Nueva referencia:", eeg.referencia)

# Cambiar a un canal válido por índice
eeg.cambiar_referencia_a(1)
print("Referencia luego de index:", eeg.referencia)

# Intentar con un nombre incorrecto
try:
    eeg.cambiar_referencia_a("Z9")
except Exception as e:
    print("Error esperado con nombre inválido:", e)

# Intentar con un índice fuera de rango
try:
    eeg.cambiar_referencia_a(999)
except Exception as e:
    print("Error esperado con índice inválido:", e)
```

✓ 0.1s

```
>> Prueba cambiar_referencia_a()
Referencia inicial: laplaciano
Nueva referencia: referenciado a C1
Referencia luego de index: referenciado a C2
Error esperado con nombre inválido: Canal 'Z9' no existe en los canales disponibles.
Error esperado con índice inválido: Índice de canal fuera de rango: 999
```

Figura 12: *Prueba del método set_reference_to().*

```python
# ======================== Método aplicar_laplaciano() ========================
print("\n>> Prueba aplicar_laplaciano()")

# Aplicar Laplaciano
try:
    eeg.aplicar_laplaciano()
    print("Filtro Laplaciano aplicado con éxito.")
    print("Referencia actual:", eeg.referencia)
except Exception as e:
    print("Error al aplicar Laplaciano:", e)
```

✓ 0.2s

```
>> Prueba aplicar_laplaciano()
Filtro Laplaciano aplicado con éxito.
Referencia actual: laplaciano
```

Figura 13: *Prueba del método apply_laplacian().*

Clase ECGSignal

En la figura 14 se muestra la creación del objeto ECGSignal. En la figura 15 se invocó detectar_picos_r() usando distintos umbrales, obteniéndose listas de índices almacenadas en picos_r. A continuación, al llamar graficar_hilbert() como se muestra en la figura 16, se obtuvo la envolvente de Hilbert de la señal.

```python
# ======================== Cargar datos y crear objeto ECGSignal ========================

data_ecg = np.load("ecg/ecg/ecg.npy")

sfreq = 512

canales = ["C1"]

info_ecg = Info(
    ch_names= canales,
    ch_types= "ecg",
    sfreq= sfreq,
    description="Registro ECG",
    experimenter="Dr. CARDIO",
    subject_info={"edad": 45, "sexo": "F"}
)

eventos_ecg = Anotaciones.load("ecg/ecg/eventos_ecg.csv", sfreq= sfreq)

ecg = ECGSignal(data=data_ecg, sfreq=sfreq, info=info_ecg, first_samp=0, anotaciones=eventos_ecg)
```

Figura 14: *Inicialización de clase ECGSignal.*

```
# ======================== Método detectar_picos_r() ========================
print("\n>> Prueba: detectar_picos_r()")
picos = ecg.detectar_picos_r(canal=0, umbral=0.6)
print(f"Picos detectados: {len(picos)}")
print("Primeros 10 picos:", picos[:10])
```

✓ 0.0s

```
>> Prueba: detectar_picos_r()
Picos detectados: 1929
Primeros 10 picos: [ 222  959 1135 1293 1500 1656 1844 2161 2464 2758]
```

Figura 15: *Prueba del método detectar_picos_r()*

```
# ====================== Método graficar_hilbert() ======================
print("\n>> Método graficar_hilbert()")

# Envolvente de Hilbert de todos los canales
ecg.graficar_hilbert()

# Envolvente de Hilbert de canales específicos
ecg.graficar_hilbert(channels=[0])
```

✓ 0.9s

```
> Método graficar_hilbert()
```

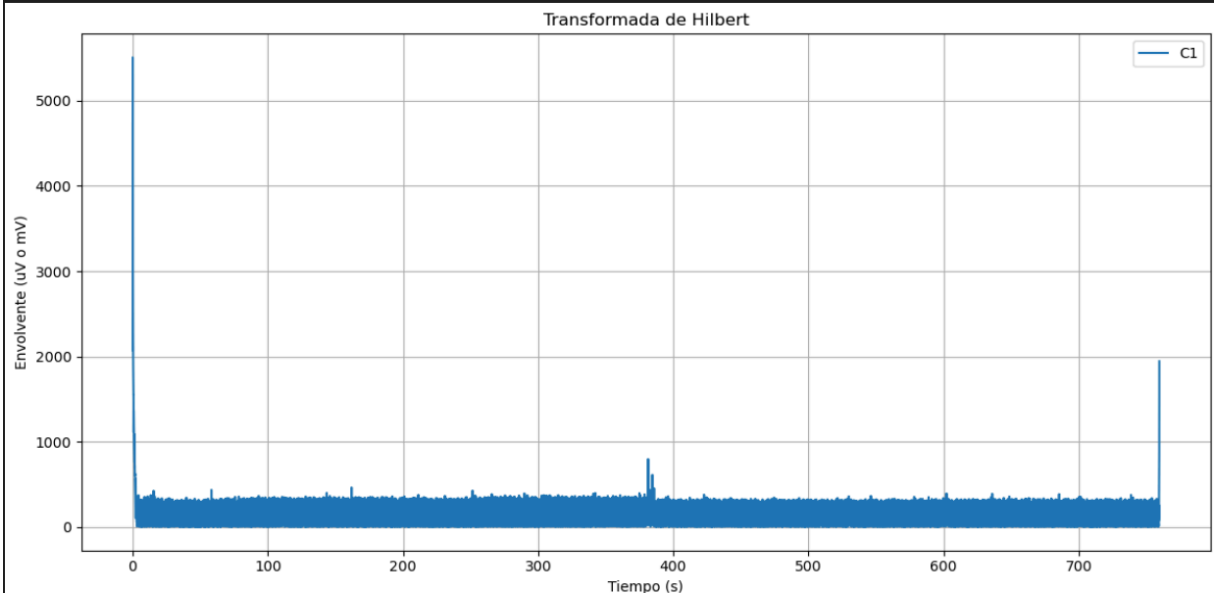


Figura 16: *Prueba del método graficar_hilbert().*

Clase EMGSignal

En la figura 17 se creó un objeto EMGSignal con datos de EMG. Se probaron sus métodos principales, en la figura 18 se invoco a detectar_activaciones(), que recorrió cada canal y registró los índices de activación donde la señal superó el umbral; en la figura 19 a calcular_nivel_rms(), que calculó el nivel RMS de la señal por ventanas; y en la figura 20 graficar_tiempo_frecuencia(), que generó un espectrograma de un canal.

```python
# ======================== Cargar datos y crear objeto EMGSignal ========================

data_emg = np.load("emg/emg/emg.npy")

sfreq = 512

canales = ["C1"]

info_emg = Info(
    ch_names= canales,
    ch_types= "emg",
    sfreq= sfreq,
    description="Registro EMG durante ejercicio",
    experimenter="Lic. KINÉSICO",
    subject_info={"edad": 28, "sexo": "F"}
)

eventos_emg = Anotaciones.load("emg/emg/eventos_emg.csv", sfreq= sfreq)

emg = EMGSignal(data=data_emg, sfreq=sfreq, info=info_emg, first_samp=0, anotaciones=eventos_emg, umbral=0.05)
```

Figura 17: *Inicialización de clase EMGSignal.*

```python
# ======================== Método detectar_activaciones() ========================
print("\n>> Método detectar_activaciones()")

activaciones = emg.detectar_activaciones()

for idx, act in enumerate(activaciones):
    print(f"Canal {canales[idx]}: {len(act)} activaciones detectadas")
```

✓ 0.0s

```
>> Método detectar_activaciones()
Canal C1: 388120 activaciones detectadas
```

Figura 18: *Prueba del método detectar_activaciones().*

```
# ======================== Método calcular_nivel_rms() ========================
print("\n>> Método calcular_nivel_rms()")
rms_resultados = emg.calcular_nivel_rms(canales=["C1"], ventana=1.0)
print("Resultados RMS por ventana:", rms_resultados)
```

✓ 0.0s

```
>> Método calcular_nivel_rms()
Resultados RMS por ventana: {'C1': array([4.60401697e+03, 2.49696171e+03, 3.02083662e+03, 1.58096738e+03,
       4.92779946e+02, 1.33632251e+03, 1.56751390e+03, 1.18362605e+03,
       5.47402157e+02, 1.63051339e+02, 4.41713388e+02, 5.53268648e+02,
       5.00929895e+02, 6.14206237e+02, 2.29004842e+02, 5.17972702e+02,
       5.05856618e+02, 1.76507130e+02, 1.29138453e+02, 2.66890705e+02,
       2.67586625e+02, 1.43463494e+02, 3.81986494e+01, 1.22044003e+02,
       1.42258879e+02, 1.06786638e+02, 6.09024202e+01, 1.44770971e+01,
       3.44560391e+01, 4.70364701e+01, 3.22230885e+01, 2.31293507e+01,
       1.06588528e+01, 8.98585635e+00, 8.96776743e+00, 1.71672441e+01,
       1.42649101e+01, 1.14505367e+01, 1.09996436e+01, 1.88323344e+01,
       3.23524147e+01, 7.75245787e+00, 2.06155189e+01, 1.30969126e+01,
       2.06318611e+01, 2.59521122e+01, 2.87346885e+01, 4.06912029e+01,
       1.93259613e+01, 6.08343850e+00, 2.52562562e+01, 3.33246760e+01,
       2.25746687e+01, 1.01035535e+02, 1.36650675e+02, 4.44214571e+01,
       2.30511448e+01, 3.93829015e+01, 1.74842289e+01, 6.97324097e+00,
       5.36275747e+00, 1.58201160e+01, 1.08182442e+01, 6.76170562e+00,
```

Figura 19: *Prueba del método calcular_nivel_rms().*

```
# ======================== Método graficar_tiempo_frecuencia() ========================
print("\n>> Método graficar_tiempo_frecuencia()")

# Espectrograma de un canal
emg.graficar_tiempo_frecuencia(canal=0, nperseg=256, noverlap=128, cmap="plasma")
```

✓ 3.6s

```
>> Método graficar_tiempo_frecuencia()
```

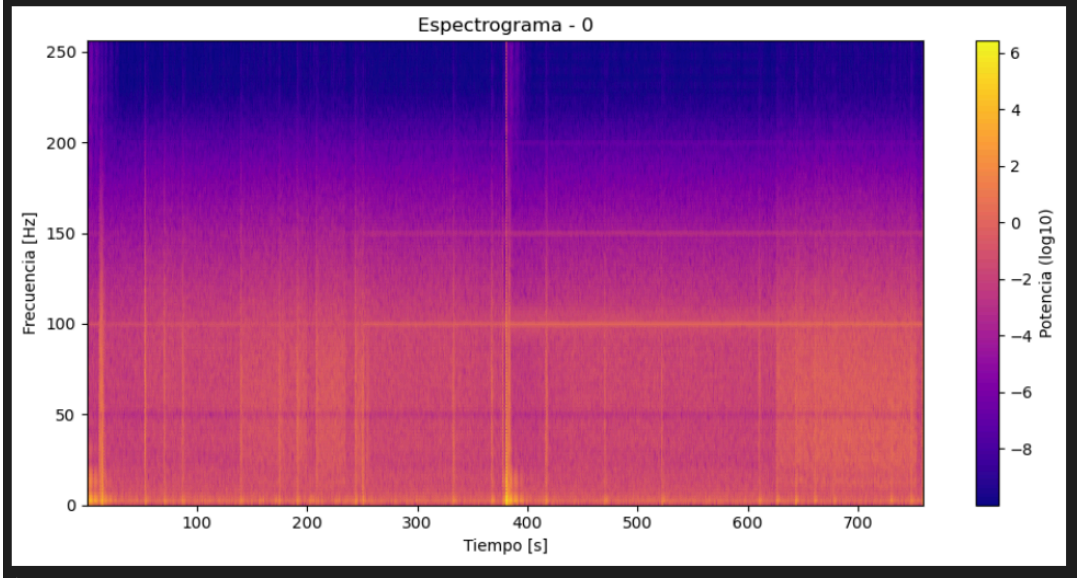


Figura 20: *Prueba del método graficar_tiempo_referencia().*

Módulo Utils

Para garantizar que los métodos de las clases reciban valores del tipo correcto, se realizaron pruebas usando la función check_types(), que verifica si cada valor coincide con los tipos esperados y lanza un TypeError en caso contrario.

Se muestra un ejemplo en la figura 21 de uso donde se capturan errores intencionados al crear un objeto Anotaciones y en la figura 22 se muestran los errores.

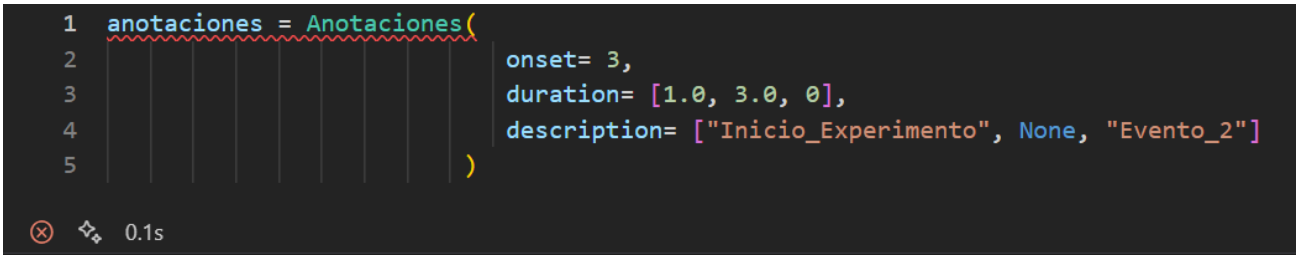


Figura 21: *Prueba del método graficar_tiempo_referencia().*

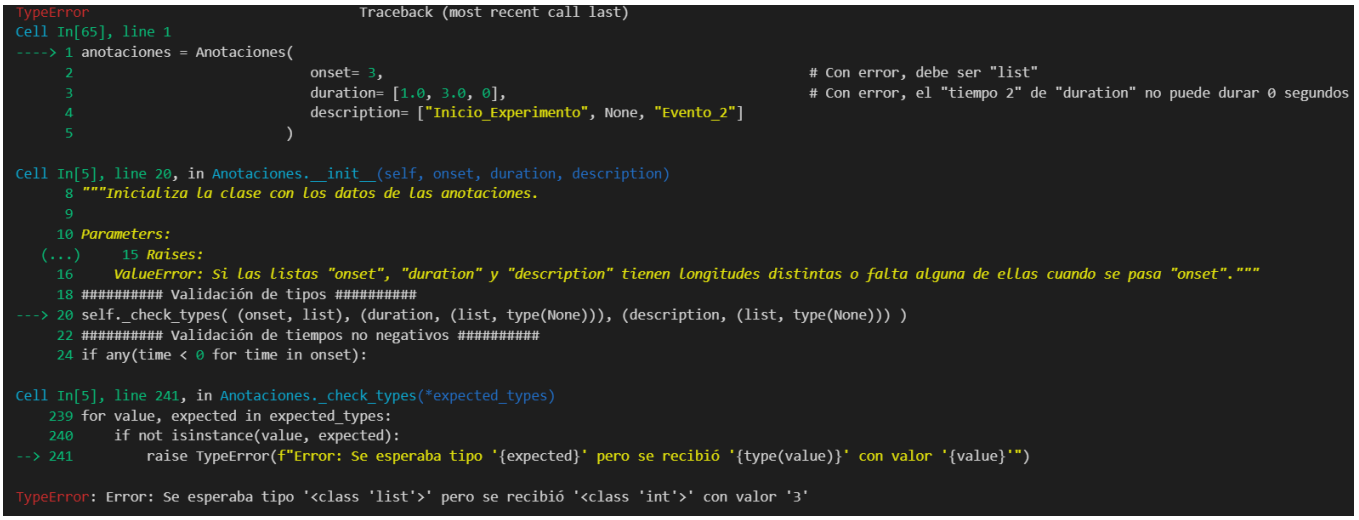


Figura 22: *Prueba de la inicialización de clase Anotaciones.*

6 Conclusiones

Tras la implementación de las clases Info, Anotaciones, RawSignal, EEGSignal, ECGSignal, EMGSignal y del módulo utils, se ha logrado una librería modular y coherente para el procesamiento de señales fisiológicas en Python. Cada componente cumple una función clara: Info gestiona metadatos, Anotaciones administra eventos temporales, RawSignal ofrece operaciones básicas de manejo de datos y las subclases especializadas (EEGSignal, ECGSignal, EMGSignal) incorporan métodos específicos de análisis (referencia espacial, detección de picos, activaciones musculares, transformadas de Hilbert, espectrogramas y métricas clínicas como frecuencia cardíaca y RMS). El módulo utils refuerza la robustez general al validar tipos, normalizar señales y convertir tiempos a muestras.

Se realizaron pruebas unitarias similares a la que se muestran en la sección analisis y resultados donde se demostraron que los métodos reaccionan correctamente ante datos válidos e invalidan apropiadamente casos erróneos mediante excepciones controladas. La arquitectura basada en POO facilita la extensibilidad—nuevas señales o métodos pueden incorporarse heredando de RawSignal—y la mantenibilidad, al centralizar las responsabilidades en clases bien delimitadas.